NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

COURSE PROJECT REPORT · COCSC741–IRS

Document Indexing and Search Engine

Assignment 1 — Mini document search engine with inverted and N-gram indexing techniques.

Harshdip Saha

Roll No. 2023UCA1897

Course: Information Retrieval Systems (COCSC741–IRS)

Submitted to
Department of Computer Science & Engineering
Academic Session 2026–27

EXECUTIVE SUMMARY

Project overview

Retrieval Desk is a local, web-based document search engine that turns a collection of text documents into two searchable indexes. It demonstrates the core information-retrieval lifecycle: document ingestion, normalization, indexing, retrieval, ranking, and performance comparison.

> **Live application:** https://document-indexing-search-engine.vercel.app
> **Source repository:** https://github.com/HarshdipSaha/document-indexing-search-engine

Objectives

1 Index documents

Accept manually entered documents and multiple UTF-8 text files, then persist and index them automatically.

2 Retrieve relevant sources

Use an inverted index for exact processed-keyword retrieval and frequency-based ranking.

3 Support partial words

Use character N-grams to locate words containing a user-entered fragment.

4 Compare techniques

Show result sets, term frequencies, and median search time for both indexing methods.

Technology stack

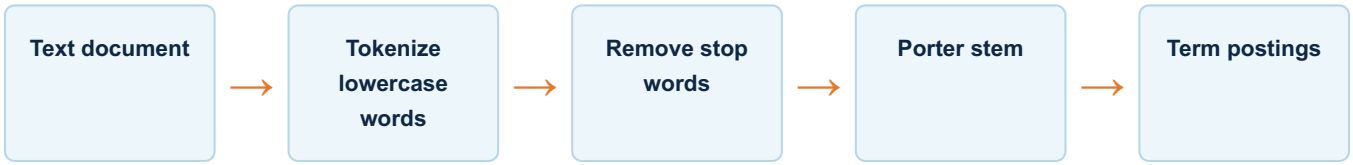
Layer	Technology	Responsibility
Application	Python + Flask	Routes, upload validation, JSON API and local server
Text processing	NLTK Porter Stemmer	Tokenization, stop-word removal and stemming
Search layer	In-memory Python dictionaries	Inverted postings, word postings and N-gram postings
Interface	HTML, CSS and JavaScript	Search, Library and Compare views with accessible results
Storage	JSON document collection	Atomic persistence; indexes rebuild at application start

The implementation is intentionally educational and local-first: it requires no external search service, API key or downloaded NLTK corpus.

Q1 · MINI DOCUMENT SEARCH ENGINE

Inverted indexing and preprocessing

The system represents each document as normalized terms and builds an **inverted index**: a mapping from each stemmed term to the document IDs where it occurs, along with its frequency in each document.



Preprocessing pipeline

Stage	Purpose	Example
Tokenization	Extract lowercase alphanumeric tokens while splitting punctuation and underscores.	“Cats, running!” → `cats, running`
Stop-word removal	Remove common function words that usually carry little retrieval value.	`the, and, were` removed
Stemming	Conflate related inflected forms so they match a shared root.	`cats → cat`, `running → run`

Index representation

```
inverted_index = {
  "comput": {"doc-01": 2, "doc-04": 1},
  "retriev": {"doc-02": 3},
  "index": {"doc-01": 1, "doc-03": 2}
}
```

The count in each posting is the term frequency. During search, postings for distinct query terms are merged using OR semantics; documents are ranked by the sum of matched query-term frequencies. Ties are resolved deterministically by title and ID.

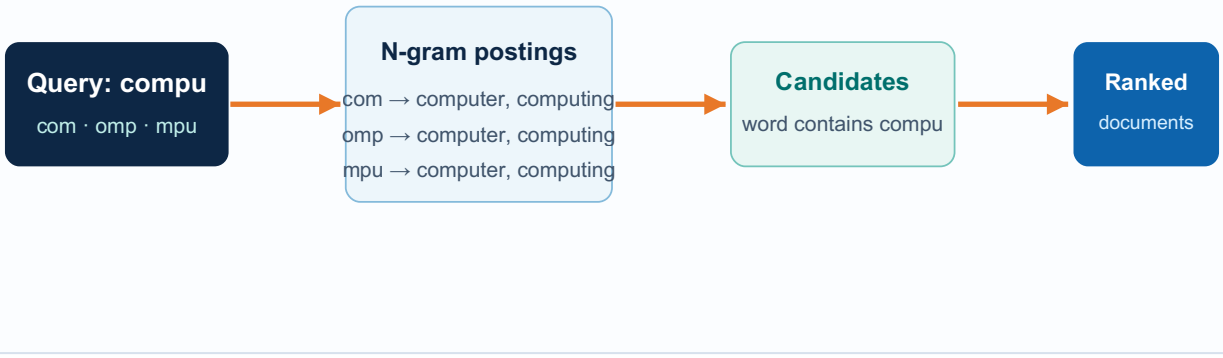
Automatic index maintenance

- Manual entry and batch upload both validate document title/text, persist the updated collection, and index the new material under a re-entrant lock.
- Writes use a temporary JSON file followed by replacement, reducing the risk of a partially written store.
- New-document API responses include updated index statistics; the client refreshes active results and collection metrics immediately.

Q2 · MULTIPLE INDEXING TECHNIQUES

N-gram retrieval and comparison

Alongside the inverted index, the application stores an unstemmed word index and a character N-gram index. This makes partial-word search possible without scanning every document body.



How partial matching works

1. Keep the raw lowercase words (without stemming) and record their document frequencies.
2. Generate trigrams for a fragment, or unigrams/bigrams for very short fragments.
3. Intersect the N-gram posting sets to identify candidate words.
4. Verify that the full fragment occurs contiguously in each candidate word, eliminating false positives.
5. Aggregate candidate-word frequency by document and rank the result list.

Technique comparison

Aspect	Inverted index	N-gram index
Query form	Exact stemmed keyword(s)	Literal partial word / fragment
Best suited to	Precise term lookups	Prefix/infix exploration
Stored key	Normalized stem	Character sequence → candidate words
Example	`computer` → stem `comput`	`compu` → computer, computers, computing
Trade-off	Fast direct lookup; misses incomplete words	Flexible partial search; intersects candidate sets

SEARCH RESULTS AND PERFORMANCE

Evaluation methodology

Every submitted query is executed against both indexes. The workbench displays the matching documents, each processed query-term frequency, total score, result counts, and median lookup time. This enables a direct and reproducible comparison.

50
warm in-process executions per index

Median
reported to reduce transient timing noise

2
indexes evaluated for every query

Query types to demonstrate

Query type	Example query	Expected observation
Exact keyword	`computer`	Inverted index retrieves documents containing the normalized stem; N-gram mode finds literal words containing the sequence.
Partial word	`compu`	Exact index generally has no posting for the incomplete term; N-gram index retrieves computer-related words.
Multiple terms	`search engine`	Both methods combine distinct terms with OR semantics; result cards show per-term frequencies and a total score.
Short fragment	`ex`	Bigram support enables partial retrieval when trigrams are not available.
Missing term	`xyznotfound`	Both result lists are empty, while the comparison view still records timing.

Presentation in the application

- **Search view:** toggles between Inverted Index and N-gram Index result lists without rerunning the query.
- **Index snapshot:** shows the current exact/partial result count and median milliseconds.
- **Comparison ledger:** retains the latest measurement per query and shows exact matches, partial matches, inverted time, N-gram time, and shared documents.
- **Document coverage table:** identifies which documents each technique retrieved and their total frequencies.
- **Accessible summary:** communicates both result counts and timings to assistive technology through a live region.

> **Interpretation:** The timings are controlled microbenchmarks of preprocessing, lookup, result construction and ranking. They exclude network, rendering, initial warm-up, disk writes and index construction; hence small timing differences should not be over-interpreted.

IMPLEMENTATION QUALITY

Validation, testing and deployment

Input validation and robustness

Upload safety

Only non-empty UTF-8 `.txt` files are accepted. A batch is fully validated before any document is added, preventing partial success for invalid input.

Request boundaries

Uploads are limited to 30 files and 5 MB; queries are limited to 500 characters. Malformed payloads return JSON errors.

Storage integrity

Document storage validates JSON at startup and uses atomic replacement on writes. Duplicate or invalid records are rejected.

Client safety

Source content is inserted through DOM text nodes rather than unsafe HTML, and cross-origin mutations are rejected.

Automated verification

Area	Verified behavior
Preprocessing	Tokenization, stop-word removal, stemming, public preprocessing module, and N-gram generation.
Exact retrieval	Frequency tracking, ranking, multiple terms, duplicate query normalization, empty queries and persistence.
Partial retrieval	Substring matching, short fragments, candidate verification, and stop-word-like fragments.
HTTP API	Document addition, index statistics, batch uploads, atomic validation, limits, JSON errors and cross-origin protection.
Interface contract	Search, Library, Compare, accessible dialogs, frequency display and live comparison summary.

✓ The project test suite contains 23 automated tests. It can be executed with:

```
python -m unittest discover -s tests -p "test_*.py" -v
python -m compileall -q app.py preprocessing.py search_engine.py tests
```

Deployment and reproducibility

The deployed application is available at https://document-indexing-search-engine.vercel.app. The repository contains setup instructions, requirements, source code, tests, and a detailed explanation of both indexing techniques. For local execution:

```
python -m pip install -r requirements.txt
python app.py
# Open http://127.0.0.1:5000
```

CONCLUSION

Assignment compliance

The project delivers a complete mini document search engine and extends it with a second indexing method for partial-word search. It is designed to make the behaviour of both methods observable rather than treating indexing as a black box.

Assignment requirement	Implementation evidence	Status
Upload/add multiple text documents	Manual add form and multi-file UTF-8 upload endpoint	**Completed**
Inverted index mapping words to documents	Stem-to-document-frequency postings in `SearchEngine`	**Completed**
Tokenization, stop words and stemming	Dedicated `preprocessing.py` module with Porter stemmer	**Completed**
Show matching documents and term frequency	Ranked result cards and coverage table	**Completed**
Automatic index update	Immediate indexing, stats response and UI refresh	**Completed**
Inverted + N-gram techniques	Exact and partial result modes	**Completed**
Exact and partial-word search	Stemmed exact queries and character-N-gram fragments	**Completed**
Result/time comparison table or visualization	Comparison ledger, snapshot and accessible live summary	**Completed**

> **Repository:** https://github.com/HarshdipSaha/document-indexing-search-engine
> **Live URL:** https://document-indexing-search-engine.vercel.app

Future scope

Potential extensions include TF–IDF/BM25 ranking, phrase and Boolean operators, highlighted matches, document deletion/editing, SQLite persistence, and evaluation on larger corpora. These enhancements can be added without changing the core separation between preprocessing, indexing, retrieval and presentation.

Submitted by: Harshdip Saha | **Roll No.:** 2023UCA1897